

Master program
"Advanced Analytics for Business"

Advanced Visual Text Analytics – Natural Language Processing



Universitatea Națională de Știință și Tehnologie Politehnica București
Facultatea de Automatică și Calculatoare



Course 3 Naïve Bayes for Sentiment Analysis



Lecture content



- What is *Naïve Bayes*?
 - Probability overview.
 - Naïve Bayes definition.
 - Laplacian smoothing.
 - Log likelihood.
 - Training.
 - Testing.
 - Assumptions and limitations.
- What are the steps for *Sentiment Analysis* using *Naïve Bayes*?
 - Build corpus.
 - Preprocessing and word count (frequency).
 - Features probability.
 - Get lambda.
 - Get the log prior.
- Python functions.

- Let's define the corpus of mixed Positive and Negative sentences.

corpus of texts

	Positive			
	Negative			

sentences containing "great"

	Positive			
	"great"			
	Negative			

If s a positive sentiment, then $P(s) = P(\text{positive}) = \frac{N_{\text{pos}}}{N} = \frac{13}{20}$

If g a sentiment containing "great" then $P(g) = \frac{N_{\text{great}}}{N} = \frac{4}{20}$

Probability of intersection: $P(s \cap g) = P(s, g) = \frac{3}{20}$

$P(\text{negative}) = 1 - P(\text{positive}) = \frac{7}{20}$

- We can define the *conditional probability* as:

sentences containing "great"

Positive	"great"			
	Negative			

$$P(\text{Positive} | \text{"great"}) = P(s|g) = \frac{3}{4}$$

$$P(\text{"great"} | \text{Positive}) = P(g|s) = \frac{3}{13}$$

How do we interpret it: *"The probability of a sentiment to be positive if it contains word "great".*

How do we interpret it: *"The probability of a positive sentence to contain the word "great".*

- We can define the *Bayes rules* as:

sentences containing "great"

Positive	"great"			
	Negative			

$$P(\text{Positive} | \text{"great"}) = P(s|g) = \frac{P(\text{Positive} \cap \text{"great"})}{P(\text{"great"})} = \frac{3}{4} \text{ - "The probability of a sentiment to be positive if it contains word "great".}$$

$$P(\text{"great"} | \text{Positive}) = P(g|s) = \frac{P(\text{"great"} \cap \text{Positive})}{P(\text{Positive})} = \frac{3}{13} \text{ The probability of a positive sentence to contain the word "great"}$$

$$P(\text{Positive} | \text{"great"}) = P(\text{"great"} | \text{Positive}) \times \frac{P(\text{Positive})}{P(\text{"great"})}$$



Bayes rules



- Example of how can we apply *Bayes rules* in sentiment analysis:
- Hypothesis:
 - Let's suppose that in one corpus, **20%** of positive sentences contain the word “great”. We know that a total of **10%** of the sentences from corpus contain word “great”, and that **40%** of the total number of sentences are positive (unbalanced data set).
- We have the new sentence like: “*Great to learn Python programming for Deep Learning.*” What is the probability that this sentence to be positive?

$$P(Positive|"great") = P("great"|Positive) \times \frac{P(Positive)}{P("great")} = 0.2 * \frac{0.4}{0.10} = 80\%$$



Naïve Bayes definition



- Example of how can we apply *Bayes rules* in sentiment analysis:
- Hypothesis:
 - Let's suppose that in one corpus, **20%** of positive sentences contain the word “great”. We know that a total of **10%** of the sentences from corpus contain word “great”, and that **40%** of the total number of sentences are positive (unbalanced data set).
- We have the new sentence like: “*Great to learn Python programming for Deep Learning.*” What is the probability that this sentence to be positive?

$$P(Positive|"great") = P("great"|Positive) \times \frac{P(Positive)}{P("great")} = 0.2 * \frac{0.4}{0.10} = 80\%$$

For a *balanced* set of positive and negative texts like:

Positive:

I am delighted to develop deep learning algorithms.

I am happy.

And **negative:**

I am unhappy, as I cannot improve my programming skills.

I feel awful.

Define $p(w_i|class)$ for each i in N_class .

E.g.: $p("am"|positive) = \frac{2}{10}$ and $p("am"|negative) = \frac{1}{11}$

vocabulary	Positive Freq	Negative Freq
I	2	3
am	2	1
delighted	1	0
develop	1	0
deep	1	0
learning	1	0
algorithms	1	0
happy	1	0
unhappy	0	1
cannot	0	1
improve	0	1
programming	0	1
skills	0	1
feel	0	1
awful	0	1
N_class	10	11





Naïve Bayes for Sentiment Analysis



For a new sentence: *I am happy to learn Python.*

$$\text{Naïve Bayes: } \prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)} = \frac{0.2}{0.27} * \frac{0.2}{0.09} * \frac{0.1}{0} * \frac{0.1}{0} \quad ???$$

For numerical reason and division by 0 we'll introduce *Laplacian smoothing*.

$p(w_i class)$		
Word	Positive	Negative
I	0.2	0.27
am	0.2	0.09
delighted	0.1	0
develop	0.1	0
deep	0.1	0
learning	0.1	0
algorithms	0.1	0
happy	0.1	0
unhappy	0	0.09
cannot	0	0.09
improve	0	0.09
programming	0	0.09
skills	0	0.09
feel	0	0.09
awful	0	0.09

For a new sentence: *I am happy to learn Python.*

Naïve Bayes: $\prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)} = \frac{0.2}{0.27} * \frac{0.2}{0.09} * \frac{0.1}{0} * \frac{0.1}{0} ???$

Laplacian smoothing:

Instead of using $P(w_i|class) = \frac{freq(w_i|class)}{N_{class}}$ we'll introduce:

$$P(w_i|class) = \frac{freq(w_i|class)+1}{N_{class}+V_{class}}, \text{ where}$$

N_{class} = frequency of all words in class;

V_{class} = number of unique words in class;

Naïve Bayes: $\prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)} = \frac{0.05}{0.05} * \frac{0.05}{0.045} * \frac{0.047}{0.041} * \frac{0.047}{0.041} = 1.46 > 1 \text{ (positive)}$

$p(w_i|class)$

Word	Positive	Negative
I	0.2	0.27
am	0.2	0.09
delighted	0.1	0
develop	0.1	0
deep	0.1	0
learning	0.1	0
algorithms	0.1	0
happy	0.1	0
unhappy	0	0.09
cannot	0	0.09
improve	0	0.09
programming	0	0.09
skills	0	0.09
feel	0	0.09
awful	0	0.09



Log Likelihood



Naïve Bayes inference:

$$\frac{P(pos)}{P(neg)} \prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)}$$

Using logarithmic property: $\log(a * b) = \log(a) + \log(b)$ then:

$$\log\left(\frac{P(pos)}{P(neg)} \prod_{i=1}^n \frac{P(w_i|pos)}{P(w_i|neg)}\right) = \log \frac{P(pos)}{P(neg)} + \sum_{i=1}^n \log \frac{P(w_i|pos)}{P(w_i|neg)}$$

log prior log likelihood

! For balanced corpus (#positive = #negative) log prior = 0.

We'll introduce: $\lambda(\omega) = \log \frac{P(\omega|pos)}{P(\omega|neg)}$ for each word in vocabulary.



Naïve Bayes training



Steps that should be followed to train a Naïve Bayes:

1. Collect and annotate corpus.
2. Data set preprocessing:
 - 2.1. lowercase.
 - 2.2 remove punctuation, urls, etc.
 - 2.3 remove stop words.
 - 2.4 stemming & tokenization.
3. Feature probability / word count. Apply Laplacian smoothing.
4. Get lambda (log likelihood).
5. Get log prior.



Naïve Bayes testing



Steps that should be followed to test a Naïve Bayes:

1. From training we get lambda (likelihood dictionary). $\lambda(w) = \log \frac{P(w|pos)}{P(w|neg)}$
2. We get logprior computed like: $\log \frac{D_{pos}}{D_{neg}}$
3. For a new sentence, e.g. *I like to learn Python*, we'll compute the score.
4. If the **score > 0**, then positive sentiment.

Causes of misclassification:

Removing punctuation: Sentence: Our beloved father... 😊 -> [belov, father]

Removing words/stop words: Sentence: This situation is not good, as your feedback is not close of being positive. -> [situation, good, feedback, close, positive].



Naïve Bayes assumptions and limitations.



1. Naïve Bayes makes the assumption that all features/words are independent.

E.g. *“It is hot during summer vacation.”* or *“It’s most of the time cold and snow during”* - [summer, spring, fall, winter] are equally weighted and a Naïve Bayes will complete the sentences with the same probability.

2. Naïve Bayes is sensitive to relative frequencies in the corpus or *unbalanced positive vs negative classes*.



Python functions



- Import library NLTK:
 - NLTK –Natural Language Toolkit - a leading platform for building Python programs to work with human language data. Ease of use and understanding: <https://www.nltk.org/book/>.
 - Text Corpora and Lexical Resources:

```
import nltk
nltk.download()

showing info https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml
2025-01-01 20:30:49.203 python[26036:1591187] +[IMKClient subclass]: chose IMKClient_Modern
2025-01-01 20:30:49.203 python[26036:1591187] +[IMKInputSession subclass]: chose IMKInputSession_Modern
True
```

- Examples of samples data for Sentiment Analysis:
 - movie_review.
 - pro_cons.
 - twitter_samples.

! It is your choice to use whatever dataset you want. Analyze and document it first.

- Import library NLTK:
 - *PorterStemmer*- stemming is the process of producing morphological variants of a root/base word. Ex: *PorterStemmer* available in nltk library.
 - Tokenize – divide strings into list of substrings. Can be used to find words and punctuation in a string.
 - E.g.:

Sentence tokenization in word_tokenize:

```
>>> s11 = "I called Dr. Jones. I called Dr. Jones."
>>> word_tokenize(s11)
['I', 'called', 'Dr.', 'Jones', '.', 'I', 'called', 'Dr.', 'Jones', '.']
>>> s12 = ("Ich muss unbedingt daran denken, Mehl, usw. fur einen "
...       "Kuchen einzukaufen. Ich muss.")
>>> word_tokenize(s12)
['Ich', 'muss', 'unbedingt', 'daran', 'denken', ',', 'Mehl', ',', 'usw',
',', 'fur', 'einen', 'Kuchen', 'einzukaufen', '.', 'Ich', 'muss', '.']
```

- *Stopwords*: - a set of commonly used words (e.g.: “the”, “is”, “and”, “when” etc).

```
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import TweetTokenizer
```




Python functions



- Import library *np*:
 - NumPy –offers comprehensive mathematical functions, linear algebra etc. Ease of use and understanding:
<https://numpy.org/>.
- Import library *pandas*:
 - Pandas is a fast powerful, flexible and easy of use open source data analysis and manipulation tool:
<https://pandas.pydata.org/>

```
import numpy as np
import pandas as pd
from nltk.corpus import movie_reviews
```

- Auxiliary library *os*:
 - *Miscellaneous operating system interfaces – portable way of using OS dependent functionality (read-write files, manipulate paths, create temp files etc).*
 - Example: `from os import getcwd`



Python functions



- Build a function for *text processing*:
 - Inputs: a string/text;
 - Outputs: a list of “clean” words containing initial text;
 - Initialize a PorterStemmer;
 - Initialize a *stopword* list – e.g. English language;
 - Use a series of methods to remove special characters;
 - using *regular expressions* remove hashtags, hyperlinks, email addresses, etc;
 - Initialize a tokenizer and then create the list of *tokens*;
 - Create a *loop* parsing each token from the list in which:
 - Remove stop words;
 - Remove punctuation;
 - Use stemming;
 - Return the list of “clean” words/tokens.



```
def process_text(str):
    """Process text function.
    Inputs:
        text: a string containing a text
    Output:
        tokens_clean: a list of words containing the processed string

    """
    stemmer = PorterStemmer()
    stopwords_english = stopwords.words('english')
    # remove hyperlinks
    str = re.sub(r'https?://[^\s\n\r]+', '', str)
    # remove hashtags
    # only removing the hash # sign from the word
    str = re.sub(r'#', '', str)
    # tokenize tweets
    tokenizer = TweetTokenizer(preserve_case=False, strip_handles=True,
                               reduce_len=True)
    str_tokens = tokenizer.tokenize(str)

    tokens_clean = []
    for word in str_tokens:
        if (word not in stopwords_english and # remove stopwords
            word not in string.punctuation): # remove punctuation
            stem_word = stemmer.stem(word) # stemming word
            tokens_clean.append(stem_word)

    return tokens_clean
```



- Build a function for *frequency* of positive and negative sentiments:
 - Inputs: a list of strings/texts, array of labels/sentiments (0 or 1);
 - Outputs: a dictionary of pairs of each word and label;

```
def build_freqs(strs, ys):  
    """  
    Inputs: strs: a list of strings  
           ys: an m x 1 array with the label (0 or 1) of each string  
    Outputs:  
           freqs: a dictionary of pairs of each word and label  
    """  
    yslist = np.squeeze(ys).tolist()  
    # Initialize an empty dictionary and populate it by looping over all strings  
    freqs = {}  
    for y, str in zip(yslist, strs):  
        for word in process_text(str):  
            pair = (word, y)  
            if pair in freqs:  
                freqs[pair] += 1  
            else:  
                freqs[pair] = 1  
    return freqs
```

- Load data positive and negative – *balanced classes*:

```
[11]: # read the pros and cons reviews
reviews_cons = pros_cons.raw('IntegratedCons.txt')
reviews_pros = pros_cons.raw('IntegratedPros.txt')
reviews_pros

[11]: '      <Pros>Easy to use, economical!</Pros>\n      <Pros>Digital is where it's at...down with developing film!</Pros>\n      <Pros>
>Good image quality, 3x optical zoom, macro mode, inexpensive</Pros>\n      <Pros>Awesome features/easy to use/fun/versatile/low price/C
ust SVS 2nd 2 none!</Pros>\n      <Pros>intuitive, user friendly</Pros>\n      <Pros>Simple, Flexible, Reliable</Pros>\n      <Pros>
battery life, download speed</Pros>\n      <Pros>Agfa quality in a small box, great for table top and tests</Pros>\n      <Pros>Image
quality, price, video link</Pros>\n      <Pros>No Extras to Buy, Very Flexible, High Quality Images, Good Software</Pros>\n      <Pros>
>simplicity</Pros>\n      <Pros>Comes with software, good price, excellent pictures.</Pros>\n      <Pros>Clear pics</Pros>\n      <P
ros>Price, Over all Performance, Zoom Feature</Pros>\n      <Pros>clear, srisp picture, durable camera, good desing, nice features</Pros>
>\n      <Pros>Inexpensive compared to some</Pros>\n      <Pros>Great picture quality, swivel lens, many options</Pros>\n      <Pros>
>Very simple controls and a wide range of features</Pros>\n      <Pros>Great camera, easy to use, good looking, high quality images.</Pr
os>\n      <Pros>Megapixel quality, easy to use</Pros>\n      <Pros>durable, easy to use</Pros>\n      <Pros>Sharp, crisp, pictures.
</Pros>\n      <Pros>Ease of use, quality pictures.</Pros>\n      <Pros>pic quality, smart media,price</Pros>\n      <Pros>megapixel
quality, easy to use</Pros>\n      <Pros>Easy to use, takes GREAT pictures, full of features.</Pros>\n      <Pros>Saturated, sharp ima
ges at all resolutions, twist-style body</Pros>\n      <Pros>very clear photos</Pros>\n      <Pros>Nickle Hydride batteries, ease of
use, excellent color rendition</Pros>\n      <Pros>Picture quality, Twist body</Pros>\n      <Pros>perfect shots, long-focus, easy to
use, a lot of features, good software</Pros>\n      <Pros>Easy to use, stylish, good picture quality, good price, good company.</Pros>\n
<Pros>Good quality pictures for a low price, great first camera</Pros>\n      <Pros>This camera is a nice easy to use camera.</Pros>\n
<Pros>Takes great pictures</Pros>\n      <Pros>Inexpensive and clear pictures</Pros>\n      <Pros>Camera is very easy to learn and us
e</Pros>\n      <Pros>extra film led for image viewing software is easy to use sleek design</Pros>\n      <Pros>Easy to use great
```

- split 80% train and 20% test data sets

```
#remove begin,end tags <Pros>, <Cons>
reviews_cons = reviews_cons.replace('<Cons>','').replace('</Cons>','').strip()
reviews_pros = reviews_pros.replace('<Pros>','').replace('</Pros>','').strip()
reviews_cons_list = reviews_cons.split('\n')
reviews_pros_list = reviews_pros.split('\n')
```



- Training and test sets are:

```
[105]: train_pos = reviews_pros_list[:16000]
       test_pos = reviews_pros_list[16000:20000]
       train_neg = reviews_cons_list[:16000]
       test_neg = reviews_cons_list[16000:20000]
       train_x = train_pos + train_neg
       test_x = test_pos + test_neg
```

- Create the numpy array of positive labels and negative labels.

```
[108]: # combine positive and negative labels
       train_x = train_pos + train_neg
       test_x = test_pos + test_neg
       train_y = np.append(np.ones((len(train_pos), 1)), np.zeros((len(train_neg), 1)), axis=0)
       test_y = np.append(np.ones((len(test_pos), 1)), np.zeros((len(test_neg), 1)), axis=0)
```

```
[110]: # Print the shape train and test sets
       print("train_y.shape = " + str(train_y.shape))
       print("test_y.shape = " + str(test_y.shape))

       train_y.shape = (32000, 1)
       test_y.shape = (8000, 1)
```

- Finally apply *process_text* and *build_freqs* functions.



- Train Naïve Bayes:
 - Loglikelihood and logprior initialization;
 - Calculate the number of unique words in the vocabulary (V);
 - Calculate number of positive words, negative words, vocabulary for positive words and vocabulary for negative words;
 - Calculate the total number of sentences and then positive and negative sentences;
 - Calculate logprior using formula: $\text{logprior} = \log \left(\frac{P(D_{pos})}{P(D_{neg})} \right) = \log \left(\frac{D_{pos}}{D_{neg}} \right)$
 - Calculate loglikelihood:

$$P(W_{pos}) = \frac{freq_{pos} + 1}{N_{pos} + V}$$

$$P(W_{neg}) = \frac{freq_{neg} + 1}{N_{neg} + V}$$

$$\text{loglikelihood} = \log \left(\frac{P(W_{pos})}{P(W_{neg})} \right)$$

DeepLearning.AI



- Naïve Bayes training:

```
def train_naive_bayes(freqs, train_x, train_y):  
    """  
    Inputs:  
        freqs: dictionary from (word, label)  
        train_x: a list of sentences  
        train_y: a list of labels (0,1)  
    Outputs:  
        logprior: the log prior  
        loglikelihood: the log likelihood of you Naive bayes equation  
    """  
    loglikelihood = {}  
    logprior = 0  
  
    # calculate the number of unique words in the vocabulary  
    vocab = set([pair[0] for pair in freqs.keys()])  
    V = len(vocab)  
  
    # Calculate number of positive words, negative words, vocabulary for positive words and vocabulary for negative words;  
  
    N_pos = N_neg = V_pos = V_neg = 0  
    for pair in freqs.keys():  
        # if the label is positive (greater than zero)  
        if pair[1] > 0:  
            .....  
        else:  
            .....  
  
    # Calculate number of total sentences  
    D = .....  
    # Calculate number of positive and negative sentences  
    D_pos = .....  
    D_neg = .....  
    # Calculate logprior  
    logprior = np.log(D_pos / D) - np.log(D_neg / D)  
  
    for word in vocab:  
        # get the positive and negative frequency of the word  
        freq_pos = .....  
        freq_neg = .....  
        # calculate the probability that each word is positive, and negative  
        p_w_pos = .....  
        p_w_neg = .....  
        # calculate the log likelihood of the word  
        loglikelihood[word] = np.log(p_w_pos / p_w_neg)  
  
    return logprior, loglikelihood
```




- Predict sentiment using Naïve Bayes:

```
def naive_bayes_predict(sentence, logprior, loglikelihood):  
    """  
    Inputs:  
        sentence: a string  
        logprior: a number  
        loglikelihood: a dictionary of words mapping to numbers  
    Outputs:  
        p: the sum of all the logliklihoods of each word in the tweet (if found in the dictionary) + logprior (a number)  
    """  
    # process the sentence to get a list of words  
    word_l = process_text(sentence)  
    # initialize probability to zero  
    p = 0  
    # add the logprior  
    p += logprior  
    for word in word_l:  
        # check if the word exists in the loglikelihood dictionary  
        if word in loglikelihood:  
            # add the log likelihood of that word to the probability  
            p += loglikelihood[word]  
    return p
```

DeepLearning.AI



- Test Naïve Bayes:

```
def test_naive_bayes(test_x, test_y, logprior, loglikelihood):
    """
    Input:
        test_x: A list of sentences
        test_y: the corresponding labels for the list of sentences
        logprior: the logprior
        loglikelihood: a dictionary with the loglikelihoods for each word
    Output:
        accuracy: (# of sentences classified correctly)/(total # of sentences from test set)
    """
    accuracy = 0 # return this properly

    y_hats = []
    for s in test_x:
        # if the prediction is > 0
        if naive_bayes_predict(s, logprior, loglikelihood) > 0:
            # the predicted class is 1
            y_hat_i = 1
        else:
            # otherwise the predicted class is 0
            y_hat_i = 0

        # append the predicted class to the list y_hats
        y_hats.append(y_hat_i)

    # error is the average of the absolute values of the differences between y_hats and test_y
    error = np.mean(np.abs(np.asarray(y_hats) - test_y))

    # Accuracy is 1 minus the error
    accuracy = 1-error

    return accuracy
```

DeepLearning.AI



Python functions



- Put all together to implement Sentiment Analysis using Naïve Bayes:

Good Luck and enjoy 😊

DeepLearning.AI